EMPLOYEE ATTENDANCE AND PAYROLL SYSTEM

1. INTRODUCTION

1.1 Introduction to the Project

Project Title: EMPLOYEE ATTENDANCE AND PAYROLL SYSTEM

The Employee Attendance and Payroll System is a MySQL-based relational database project developed to manage employee details, attendance records, leave information, working hours, salary structures, tax deductions, and payroll processing.

In many organizations, employee attendance and salary information may be maintained using paper registers or separate spreadsheets. Such methods can result in data duplication, calculation errors, difficulty in tracking leave, and delays in salary processing.

The proposed system addresses these problems by storing related information in interconnected database tables. Authorized users can retrieve employee details, check attendance, review leave requests, calculate salaries, and generate reports using SQL queries.

The major functions of the system are explained below.

1. Employee Management: Stores employee ID, name, contact information, department, designation, joining date, and employment status.
2. Attendance Management: Records daily attendance, check-in time, check-out time, working hours, and attendance status.
3. Leave Management: Maintains leave requests, leave types, dates, reasons, approval status, and leave balances.
4. Working Hours Management: Calculates daily and monthly working hours using attendance records.
5. Salary Management: Maintains basic salary, house rent allowance (HRA), other allowances, and deductions.
6. Tax and Deduction Management: Records applicable tax amounts and other salary deductions according to the organization's defined rules.
7. Payroll Management: Maintains monthly salary records, gross salary, total deductions, net salary, payment date, and payment status.
8. Overtime Management: Records overtime hours and calculates eligible overtime payments.
9. Report Generation: Generates employee, attendance, leave, salary, and payroll reports.

The system uses primary keys, foreign keys, unique constraints, and other database rules to maintain data integrity. SQL operations such as CREATE, INSERT, SELECT, UPDATE, and DELETE are used to manage the database.

Overall, the project demonstrates how a relational database can improve the accuracy, efficiency, security, and organization of employee attendance and payroll operations.**1.2 Purpose and Objectives**

1.2 Purpose and Objectives

1.2.1 Purpose of the Project

The main purpose of the Employee Attendance and Payroll System is to develop a centralized relational database using MySQL to manage employee information, attendance, leave requests, salary calculations, and payroll records accurately.

The project aims to reduce the limitations of manual employee management by maintaining organized and interconnected records. It helps organizations track working hours, process salary calculations, manage leave requests, and retrieve information whenever required.

The main purposes of the project are:

1. To maintain employee information in a centralized database.
2. To record daily attendance and calculate working hours.
3. To manage employee leave requests and approval status.
4. To maintain employee salary structures and allowances.
5. To calculate gross salary, taxes, deductions, and net salary.
6. To manage monthly payroll records and payment status.
7. To reduce manual paperwork and calculation errors.
8. To maintain data accuracy through database constraints.
9. To retrieve employee information using SQL queries.
10. To generate attendance, leave, and payroll reports.
11. To protect confidential employee and salary information.
12. To demonstrate the practical implementation of relational database concepts.

1.2.2 Objectives of the Project

The main objectives of the Employee Attendance and Payroll System are as follows.

1. **Employee Information Management: Maintain complete employee records, including employee ID, name, department, designation, contact details, joining date, and employment status.**
2. **Attendance Tracking: Record employee attendance dates, check-in times, check-out times, and attendance status**.
3. **Leave Management: Store leave applications, leave types, start and end dates, reasons, and approval details**.
4. **Working Hours Calculation: Calculate daily and monthly working hours from recorded attendance information.**
5. **Salary Calculation: Calculate employee salaries using basic salary, allowances, attendance, overtime, and deductions.**

6. **Tax and Deduction Calculation: Maintain applicable tax and other deduction details used in payroll processing.**
7. **Payroll Processing: Generate monthly payroll records containing gross salary, deductions, net salary, and payment status.**
8. **Overtime Calculation: Record overtime hours and calculate additional payments when employees are eligible**.
9. **Data Accuracy and Integrity: Use primary keys, foreign keys, unique constraints, and validation rules to prevent invalid records.**
10. **Efficient Data Retrieval: Retrieve employee, attendance, leave, and salary information using SQL queries and JOIN operations**.
11. **Report Generation: Generate employee attendance reports, leave summaries, salary reports, and monthly payroll summaries**.
12. **Security and Privacy: Restrict access to employee information and salary details to authorized users**.

13. **Reduced Data Redundancy: Store information in separate but related tables to avoid unnecessary duplication.**

14. **Improved Efficiency: Reduce administrative workload and support faster employee and payroll management.**

1.2.3 Expected Outcome

The expected outcome of the project is a functional relational database that stores and manages employee attendance and payroll information in an organized manner.

The project is expected to achieve the following outcomes:

1. A structured database containing employee, department, attendance, leave, salary, payroll, and user information.
2. Proper relationships between tables using primary keys and foreign keys
3. Accurate storage and retrieval of employee records.
4. Effective tracking of attendance, working hours, and leave balances.
5. Salary calculations based on defined allowances and deductions.
6. Monthly payroll records with gross salary, total deductions, net salary, and payment status.
7. Generation of useful reports using SQL queries, aggregate functions, and JOIN operations.
8. Improved understanding of DBMS concepts such as relational database design, integrity constraints, subqueries, views, triggers, and stored procedures.

The project demonstrates how a database system can simplify employee administration and support reliable attendance and payroll management.

1.3 List of Database Tables

The database design described in your Employee Attendance and Payroll System document contains seven main entities.

S.No.	Table Name	Purpose
1	Employee	Stores personal and professional employee details.
2	Department	Stores department names, descriptions, and manager information.
3	Attendance	Maintains daily attendance and working-hour records.
4	Leave	Stores employee leave requests and approval status.
5	Salary	Maintains employee salary structure and compensation details.
6	Payroll	Stores monthly payroll calculations and payment information.
7	User / Admin	Maintains login details and access roles for authorized users.

These tables represent the core entities in the supplied project document. Tax information and overtime payments can be represented through suitable payroll fields or extended into separate tables if the implementation requires more detailed tracking.

1.4 Attributes of Database Tables

Each database table contains attributes, also called columns, that describe the information stored in it.

1. **Employee Table**
   Purpose: Stores the personal and professional details of employees.
   - **Employee_ID – Primary key that uniquely identifies an employee.**
   - **First_Name – Employee's first name.**
   - **Last_Name – Employee's last name.**
   - **Gender – Employee's gender.**
   - **Date_of_Birth – Employee's date of birth.**
   - **Email – Employee's email address.**
   - **Phone_Number – Employee's contact number.**
   - **Address – Employee's residential address.**
   - **Department_ID – Foreign key referencing the department.**
   - **Designation – Employee's job position.**
   - **Date_of_Joining – Date the employee joined the organization.**
   - **Employment_Status – Indicates whether the employee is active or inactive.**

2. **Department Table**
   Purpose: Stores information about organizational departments.
   - **Department_ID – Primary key.**
   - **Department_Name – Name of the department.**
   - **Department_Description – Description of the department.**
   - **Manager_ID – Identifier of the department manager.**
3. **Attendance Table**
   Purpose: Stores daily attendance records.
   - **Attendance_ID – Primary key.**
   - **Employee_ID – Foreign key referencing the employee.**
   - **Attendance_Date – Date of attendance.**
   - **Check_In_Time – Employee's arrival time.**
   - **Check_Out_Time – Employee's departure time.**
   - **Working_Hours – Total working hours for the day.**
   - **Attendance_Status – Present, absent, half-day, or another defined status.**
4. **Leave Table**
   Purpose: Maintains employee leave requests and their approval details.
   - **Leave_ID – Primary key.**
   - **Employee_ID – Foreign key referencing the employee.**
   - **Leave_Type – Type of leave requested.**
   - **Start_Date – First day of leave.**
   - **End_Date – Last day of leave.**
   - **Number_of_Days – Total requested leave days.**
   - **Reason – Reason for requesting leave.**
   - **Request_Date – Date the request was submitted.**
   - **Approval_Status – Pending, approved, or rejected.**
   - **Approved_By – Person responsible for approving the leave.**
5. **Salary Table**
   Purpose: Stores salary structure and compensation information.
   - **Salary_ID – Primary key.**
   - **Employee_ID – Foreign key referencing the employee.**
   - **Basic_Salary – Employee's basic salary.**
   - **House_Rent_Allowance – Housing allowance.**
   - **Other_Allowances – Additional allowances.**
   - **Deductions – Applicable salary deductions.**
   - **Effective_From – Date the salary structure becomes effective.**
   - **Effective_To – Date the salary structure ends, if applicable.**
6. **Payroll Table**
   Purpose: Stores monthly salary calculations and payment information.
   - **Payroll_ID – Primary key.**
   - **Employee_ID – Foreign key referencing the employee.**
   - **Salary_ID – Foreign key referencing the salary record.**
   - **Payroll_Month – Month for which payroll is calculated.**
   - **Payroll_Year – Year of the payroll period.**

- **Total_Working_Days – Total working days in the period.**
- **Days_Present – Number of days attended.**
- **Days_Absent – Number of days absent.**
- **Overtime_Hours – Total eligible overtime hours.**
- **Gross_Salary – Salary before deductions.**
- **Total_Deductions – Total deductions from salary, including applicable taxes.**
- **Net_Salary – Final salary payable after deductions.**
- **Payment_Date – Date salary is paid.**
- **Payment_Status – Pending, paid, or another defined status.**

7. **User / Admin Table**
   Purpose: Stores login details and authorization information.
   - **User_ID – Primary key.**
   - **Employee_ID – Foreign key referencing an employee; may be NULL for an external administrator.**
   - **Username – Login username.**
   - **Password_Hash – Securely stored password hash.**
   - **Role – User role, such as administrator or authorized staff member.**
   - **Created_At – Account creation date and time.**
   - **Account_Status – Indicates whether the account is active or inactive.**

The attributes listed above follow the supplied Employee Attendance and Payroll System document. Additional tax-specific fields or a separate tax table may be introduced if individual tax types, rates, and periods need to be stored independently.

2. REQUIREMENT ANALYSIS

Requirement analysis identifies the functions that the Employee Attendance and Payroll System must perform and the quality standards it should maintain. It includes functional requirements, non-functional requirements, and integrity requirements.

2.1 Functional Requirements

Functional requirements describe the operations performed by the system.

2.1.1 Employee Management

1. The system shall store employee ID, name, department, designation, contact details, and joining date.
2. Each employee shall have a unique Employee ID
3. The system shall allow authorized users to insert, retrieve, update, and manage employee records.
4. The system shall support searching employees by ID, name, or department.

2.1.2 Attendance Management

1. The system shall record daily employee attendance.

2. It shall store check-in time and check-out time.
3. It shall maintain attendance status, such as present, absent, and half-day.
4. It shall calculate daily and monthly working hours.
5. It shall generate attendance reports for selected dates or periods.

2.1.3 Working Hours Calculation

1. The system shall calculate working hours from check-in and check-out times.
2. It shall summarize the total working hours for each employee.
3. It shall support monthly working-hour reports.
4. It shall identify recorded overtime hours.
5. It shall validate working-hour values to prevent invalid entries.

2.1.4 Leave Management

1. The system shall record employee leave requests.
2. It shall maintain leave type, start date, end date, number of days, and reason.
3. It shall track approval status, including pending, approved, and rejected.
4. It shall allow authorized personnel to review and approve leave requests.
5. It shall maintain leave history and available leave balances.

2.1.5 Salary Calculation

1. The system shall store basic salary and salary-related allowances.
2. It shall calculate gross salary using the defined salary components.
3. It shall account for applicable attendance-based adjustments and eligible overtime.
4. It shall calculate net salary after applicable deductions.
5. It shall maintain salary history and effective dates.

2.1.6 Payroll Management

1. The system shall generate monthly payroll records.
2. It shall store payroll month, year, working days, days present, and days absent.
3. It shall maintain gross salary, total deductions, and net salary
4. It shall record payment date and payment status.
5. It shall allow authorized users to retrieve payroll records for a selected month and year.

2.1.7 Tax and Deduction Management

1. The system shall record applicable tax and other deductions.
2. It shall include applicable deductions in payroll calculations.
3. It shall maintain total deductions for each payroll period.
4. It shall support retrieving employee-wise deduction details.
5. Tax calculations shall follow the rules configured for the organization and applicable tax requirements.

2.1.8 Employee Search

1. The system shall allow users to search for employees using Employee ID.

2. It shall support searching by employee name.
3. It shall display relevant employee information based on the search criteria.
4. It shall allow filtering employees by department and employment status.

2.1.9 Report Generation

1. The system shall generate daily and monthly attendance reports.
2. It shall generate leave request and leave balance reports.
3. It shall generate employee salary reports.
4. It shall support department-wise employee and payroll summaries.
5. It shall generate payroll summaries for selected months and years.

2.1.10 Overtime Management

1. The system shall record overtime hours.
2. It shall calculate eligible overtime payments using configured rules.
3. It shall include eligible overtime amounts in the relevant payroll calculations.
4. It shall support employee-wise overtime reports.

2.1.11 Leave Balance Management

1. The system shall maintain employee leave balances.
2. It shall record leave taken by each employee.
3. It shall support leave history retrieval.
4. It shall update leave balances according to approved leave and the organization's leave rules.

2.1.12 User Access Management

1. The system shall maintain user login information.
2. **It** shall assign appropriate roles to administrators and authorized users.
3. It shall restrict access to confidential salary and employee information.
4. It shall support account status management.
5. Passwords shall be stored securely as password hashes rather than plain text.

2.2 Non-Functional Requirements

Non-functional requirements define the quality, security, and operational standards of the system.

2.2.1 Performance

The system should retrieve employee details, attendance records, and salary information within a reasonable time. SQL queries should be efficient and continue to perform acceptably as records increase.

2.2.2 Reliability

The system should maintain accurate attendance and payroll records. Related operations should be handled carefully to prevent incomplete or inconsistent updates.

2.2.3 Security

Employee information and salary details must be protected against unauthorized access. Database permissions and user roles should restrict sensitive operations.

2.2.4 Usability

The system should be easy to understand and operate. Table names, attributes, and reports should be clearly organized.

2.2.5 Availability

The system should be accessible to authorized users whenever required, except during scheduled maintenance or unexpected service interruptions.

2.2.6 Data Integrity

The database should prevent duplicate identifiers, invalid references, missing mandatory values, and invalid salary or attendance entries.

2.2.7 Scalability

The database should support increasing numbers of employees, attendance records, leave requests, and payroll transactions without major structural changes.

2.2.8 **Maintainability**

The database structure, constraints, and SQL queries should be organized and documented so that future changes can be implemented easily.

2.2.9 Backup and Recovery

Regular database backups should be maintained. Suitable recovery procedures should allow records to be restored following accidental deletion or system failure.

2.2.10 Privacy

Confidential information, including employee addresses, contact details, and salary information, should be accessible only to authorized personnel.

2.2.11 Compatibility

The database should operate with the selected MySQL version and supported operating system. Any application interface should be compatible with its intended platform.

2.2.12 Accuracy

Working hours, leave balances, overtime payments, tax deductions, and net salaries should be calculated accurately according to the rules defined for the system.

2.3 Integration Requirements

Integrity requirements maintain the correctness and consistency of the database.

2.3.1 Entity Integrity

Each table must have a primary key that uniquely identifies every record.

2.3.2 Referential Integrity

Foreign keys must maintain valid relationships between employees, departments, attendance, leave, salary, and payroll records.

2.3.3 Domain Integrity

Attributes must contain values compatible with their defined data types and constraints.

2.3.4 Uniqueness

Employee IDs, usernames, and other attributes requiring unique values must not contain duplicates.

2.3.5 Data Consistency

Employee, attendance, leave, and payroll information must remain consistent across related tables.

2.3.6 Mandatory Fields

Required attributes must not accept NULL values.

2.3.7 Validity Constraints

Invalid values, such as negative salary amounts or invalid working hours, must be rejected.

2.3.8 Update Integrity

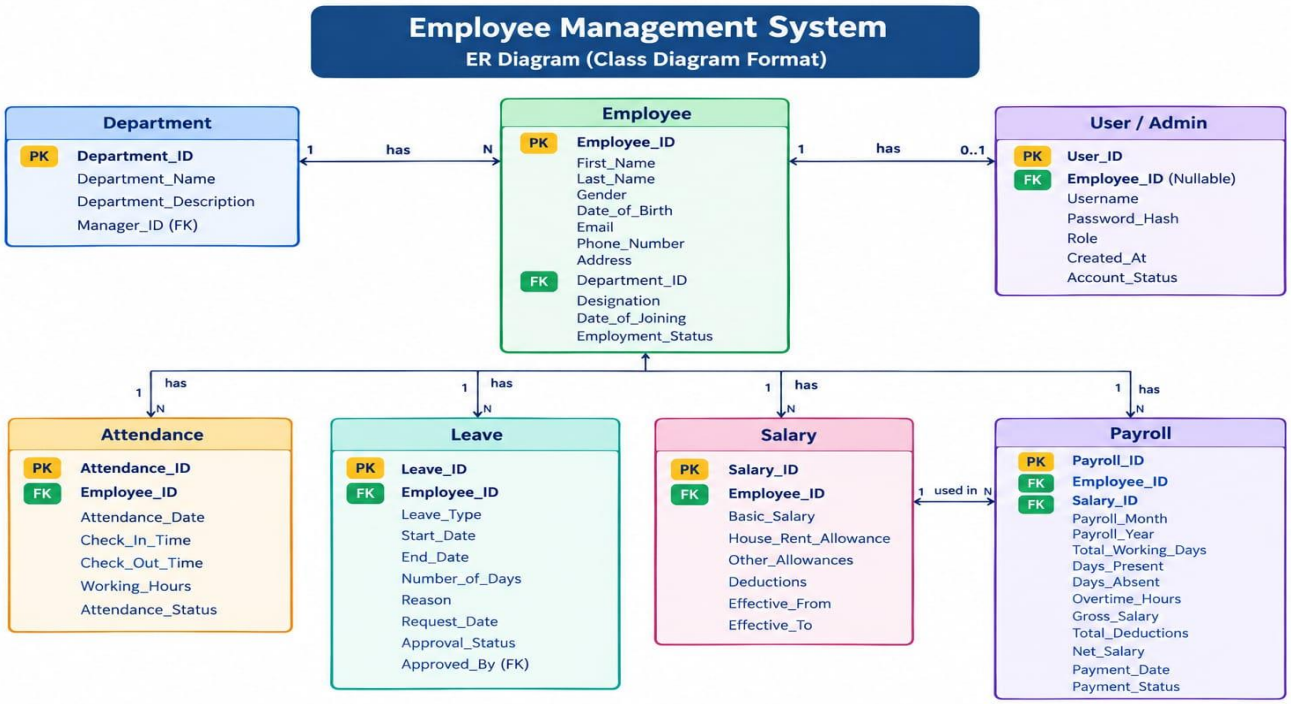
Updates to related records must preserve valid relationships and database consistency.

3. ER DIAGRAM

The Entity Relationship (ER) Diagram represents the logical structure of the Employee Attendance and Payroll System. It shows the entities, their attributes, primary keys, foreign keys, and relationships.

The ER design helps organize employee-related information into separate but interconnected tables, reducing unnecessary duplication and improving data integrity.

3.1 Entity Relationship Diagram



3.2 Entities and Key Identification

Entity	Primary key	Foreign key
Employee	Employee_ID	Department_ID
Department	Department_ID	Manager_ID, where configured as a reference
Attendance	Attendance_ID	Employee_ID
Leave	Leave_ID	Employee_ID
Salary	Salary_ID	Employee_ID
Payroll	Payroll_ID	Employee_ID,Salary_ID
User / Admin	User_ID	Employee_ID, nullable for external administrators

These keys help ensure that attendance, leave, salary, and payroll records refer to valid employees.

3.3 Relationships and Cardinalities

Cardinality describes how records in one table relate to records in another table.

1. **Category – Product:** One category can contain multiple products, while each product belongs to one category.
2. **Supplier – PurchaseOrder:** One supplier can receive multiple purchase orders, while each purchase order references one supplier.
3. **Employee – PurchaseOrder:** An employee can place multiple purchase orders, while each order is associated with one employee.
4. **PurchaseOrder – PurchaseOrderItem:** One purchase order can contain multiple order items, with each item referring to a particular product.
5. **Customer – Sale:** One customer can be associated with multiple sales. A sale may also have no registered customer when CustomerID is NULL.
6. **Employee – Sale:** One employee can handle multiple sales transactions, while each sale is associated with one employee.
7. **Sale – SaleItem:** One sale can contain multiple sale items, and each item identifies a product included in the transaction.
8. **Product – Stock – Warehouse:** Stock connects products with warehouses and maintains the quantity of each product at a specific location.
9. **Product – ProductSupplier – Supplier:** ProductSupplier connects products with suppliers and maintains supplier-specific pricing and lead-time details.
10. **Product – PriceHistory:** One product can have multiple historical price records, while each price record belongs to one product.

3.3.2 Indirect Many-to-Many Relationships

A many-to-many relationship occurs when multiple records of one entity can be associated with multiple records of another entity. These relationships are implemented using junction tables.

3.3.1 Employee–Department Relationship

One department can contain multiple employees, while each employee belongs to a department in the basic design.

Relationship: One-to-Many (1:N).

3.3.2 Employee–Attendance Relationship

One employee can have multiple attendance records for different working days. Each attendance record belongs to one employee.

Relationship: One-to-Many (1:N).

3.3.3 Employee–Leave Relationship

One employee can submit multiple leave requests. Each leave request belongs to one employee.

Relationship: One-to-Many (1:N)

3.3.4 Employee–Salary Relationship

One employee can have multiple salary records over time when salary structures change. Each salary record refers to one employee.

Relationship: One-to-Many (1:N).

3.3.5 Employee–Payroll Relationship

One employee can have multiple monthly payroll records. Each payroll record refers to one employee and the applicable salary record.

Relationship: One-to-Many (1:N).

3.3.6 Employee–User Relationship

An employee may be associated with a user account. An external administrator may have an account without an Employee ID. The exact cardinality depends on the account rules implemented.

3.4 Importance of the ER Diagram

The ER diagram provides a clear understanding of the database structure. It helps identify entities, avoid unnecessary duplication, define valid relationships, and simplify SQL JOIN operations. It also provides a foundation for creating and maintaining the database tables.

4. DATABASE IMPLEMENTATION

Database implementation is the process of converting the database design into actual MySQL tables. It includes creating the database, defining table structures, establishing relationships, inserting records, and retrieving information using SQL commands.

The Employee Attendance and Payroll System uses relational database concepts to organize employee records and manage attendance and payroll operations.

4.1 Database Creation

The first step is to create the database and select it for further operations.

SQL Command:

```
CREATE DATABASE EmployeePayrollSystem;
```

Output:

```
Query OK, 1 row affected (0.03 sec)
```

Selecting the Database:

```
USE EmployeePayrollSystem;
```

Output:

```
Database changed
```

4.2 Table Creation, Data Insertion and Verification

The database contains seven main tables. Each table stores a particular type of information and uses primary keys and foreign keys to maintain relationships.

4.2.1 Employee Table

Table Creation (DDL):

```
CREATE TABLE Department (
    Department_ID INT PRIMARY KEY,
    Department_Name VARCHAR(50) NOT NULL,
    Department_Description VARCHAR(100),
    Manager_ID INT
);
```

Data Insertion (DML):

INSERT INTO Department VALUES

(1, 'HR', 'Human Resources', 1),

(2, 'IT', 'Information Technology', 2),

(3, 'Finance', 'Finance and Accounts', 3),

(4, 'Marketing', 'Marketing Department', 4),

(5, 'Sales', 'Sales Department', 5),

(6, 'Operations', 'Operations Management', 6),

(7, 'Development', 'Software Development', 7),

(8, 'Testing', 'Software Testing', 8),

(9, 'Support', 'Customer Support', 9),

(10, 'Administration', 'General Administration', 10);

Verification Query:

```
SELECT * FROM Department;
```

Output:.

```
mysql> select * from Department;
+---------------+-----------------+-------------------------+------------+
| Department_ID | Department_Name | Department_Description | Manager_ID |
+---------------+-----------------+-------------------------+------------+
|             1 | HR              | Human Resources         |          1 |
|             2 | IT              | Information Technology |          2 |
|             3 | Finance         | Finance and Accounts    |          3 |
|             4 | Marketing       | Marketing Department    |          4 |
|             5 | Sales           | Sales Department        |          5 |
|             6 | Operations      | Operations Management   |          6 |
|             7 | Development     | Software Development    |          7 |
|             8 | Testing         | Software Testing        |          8 |
|             9 | Support         | Customer Support        |          9 |
|            10 | Administration  | General Administration |         10 |
+---------------+-----------------+-------------------------+------------+
10 rows in set (0.00 sec)
```

4.2 Employee Table

Table Creation (DDL):

```
CREATE TABLE Employee (
    Employee_ID INT PRIMARY KEY,
    First_Name VARCHAR(30) NOT NULL,
    Last_Name VARCHAR(30),
    Gender VARCHAR(10),
    Date_of_Birth DATE,
    Email VARCHAR(100) UNIQUE,
    Phone_Number VARCHAR(15),
    Address VARCHAR(100),
    Department_ID INT,
    Designation VARCHAR(50),
    Date_of_Joining DATE,
    Employment_Status VARCHAR(20),
    FOREIGN KEY (Department_ID)
        REFERENCES Department(Department_ID)
);
```

Data Insertion (DML):

```
INSERT INTO Employee VALUES
(1, 'Rahul', 'Sharma', 'Male', '1995-05-10',
 'rahul@gmail.com', '9876500001', 'Hyderabad', 1,
 'HR Manager', '2020-01-10', 'Active'),

(2, 'Priya', 'Reddy', 'Female', '1998-03-15',
 'priya@gmail.com', '9876500002', 'Hyderabad', 2,
 'Software Engineer', '2021-06-15', 'Active'),
```

```
(3, 'Amit', 'Kumar', 'Male', '1996-07-20',
 'amit@gmail.com', '9876500003', 'Vijayawada', 3,
 'Accountant', '2022-02-01', 'Active'),

(4, 'Sneha', 'Rao', 'Female', '1997-09-12',
 'sneha@gmail.com', '9876500004', 'Warangal', 4,
 'Marketing Executive', '2021-08-10', 'Active'),

(5, 'Kiran', 'Reddy', 'Male', '1994-11-25',
 'kiran@gmail.com', '9876500005', 'Guntur', 5,
 'Sales Executive', '2019-04-05', 'Active'),

(6, 'Ananya', 'Patel', 'Female', '1999-01-18',
 'ananya@gmail.com', '9876500006', 'Hyderabad', 6,
 'Operations Executive', '2023-01-12', 'Active'),

(7, 'Vikram', 'Singh', 'Male', '1995-06-30',
 'vikram@gmail.com', '9876500007', 'Nizamabad', 7,
 'Developer', '2020-09-20', 'Active'),

(8, 'Divya', 'Mohan', 'Female', '1998-12-05',
 'divya@gmail.com', '9876500008', 'Karimnagar', 8,
 'Test Engineer', '2022-07-01', 'Active'),

(9, 'Arjun', 'Verma', 'Male', '1993-04-22',
 'arjun@gmail.com', '9876500009', 'Hyderabad', 9,
 'Support Engineer', '2018-03-15', 'Active'),
```

(10, 'Meena', 'Das', 'Female', '1997-08-14',

'meena@gmail.com', '9876500010', 'Khammam', 10,

'Administrator', '2021-11-01', 'Active');

Verification Query:

```
SELECT * FROM Employee;
```

Output:

```
mysql> select * from Employee;
```

Employee_ID	First_Name	Last_Name	Gender	Date_of_Birth	Email	Phone_Number	Address	Department_ID	Designation	Date_of_Joining	Employment_Status
1	Rahul	Sharma	Male	1995-05-10	rahul@gmail.com	9876500001	Hyderabad	1	HR Manager	2020-01-10	Active
2	Priya	Reddy	Female	1998-03-15	priya@gmail.com	9876500002	Hyderabad	2	Software Engineer	2021-06-15	Active
3	Amit	Kumar	Male	1996-07-20	amit@gmail.com	9876500003	Vijayawada	3	Accountant	2022-02-01	Active
4	Sneha	Rao	Female	1997-09-12	sneha@gmail.com	9876500004	Warangal	4	Marketing Executive	2021-08-10	Active
5	Kiran	Reddy	Male	1994-11-25	kiran@gmail.com	9876500005	Guntur	5	Sales Executive	2019-04-05	Active
6	Ananya	Patel	Female	1999-01-18	ananya@gmail.com	9876500006	Hyderabad	6	Operations Executive	2023-01-12	Active
7	Vikram	Singh	Male	1995-06-30	vikram@gmail.com	9876500007	Nizamabad	7	Developer	2020-09-20	Active
8	Divya	Mohan	Female	1998-12-05	divya@gmail.com	9876500008	Karimnagar	8	Test Engineer	2022-07-01	Active
9	Arjun	Verma	Male	1993-04-22	arjun@gmail.com	9876500009	Hyderabad	9	Support Engineer	2018-03-15	Active
10	Meena	Das	Female	1997-08-14	meena@gmail.com	9876500010	Khammam	10	Administrator	2021-11-01	Active

```
10 rows in set (0.00 sec)
```

4.2.3 Attendance Table

Table Creation (DDL):

```
CREATE TABLE Attendance (
    Attendance_ID INT PRIMARY KEY,
    Employee_ID INT,
    Attendance_Date DATE,
    Check_In_Time TIME,
    Check_Out_Time TIME,
    Working_Hours DECIMAL(4,2),
    Attendance_Status VARCHAR(20),
    FOREIGN KEY (Employee_ID)
        REFERENCES Employee(Employee_ID)
);
```

Data Insertion (DML):

INSERT INTO Attendance VALUES

(1, 1, '2026-09-01', '09:00:00', '18:00:00', 8.00, 'Present'),

(2, 2, '2026-09-01', '09:15:00', '18:15:00', 8.00, 'Present'),

(3, 3, '2026-09-01', '09:00:00', '13:00:00', 4.00, 'Half-day'),

(4, 4, '2026-09-01', NULL, NULL, 0.00, 'Absent'),

(5, 5, '2026-09-01', '08:45:00', '18:45:00', 9.00, 'Present'),

(6, 6, '2026-09-02', '09:00:00', '17:00:00', 7.00, 'Present'),

(7, 7, '2026-09-02', '09:30:00', '18:30:00', 8.00, 'Present'),

(8, 8, '2026-09-02', NULL, NULL, 0.00, 'Absent'),

(9, 9, '2026-09-02', '09:00:00', '19:00:00', 9.00, 'Present'),

(10, 10, '2026-09-02', '09:00:00', '13:00:00', 4.00, 'Half-day');

Verification Query:

```
SELECT * FROM Attendance;
```

Output:

```
mysql> select * from Attendance;
+---------------+-------------+-----------------+---------------+----------------+---------------+-------------------+
| Attendance_ID | Employee_ID | Attendance_Date | Check_In_Time | Check_Out_Time | Working_Hours | Attendance_Status |
+---------------+-------------+-----------------+---------------+----------------+---------------+-------------------+
|             1 |           1 | 2026-09-01      | 09:00:00      | 18:00:00       |          8.00 | Present           |
|             2 |           2 | 2026-09-01      | 09:15:00      | 18:15:00       |          8.00 | Present           |
|             3 |           3 | 2026-09-01      | 09:00:00      | 13:00:00       |          4.00 | Half-day          |
|             4 |           4 | 2026-09-01      | NULL          | NULL           |          0.00 | Absent            |
|             5 |           5 | 2026-09-01      | 08:45:00      | 18:45:00       |          9.00 | Present           |
|             6 |           6 | 2026-09-02      | 09:00:00      | 17:00:00       |          7.00 | Present           |
|             7 |           7 | 2026-09-02      | 09:30:00      | 18:30:00       |          8.00 | Present           |
|             8 |           8 | 2026-09-02      | NULL          | NULL           |          0.00 | Absent            |
|             9 |           9 | 2026-09-02      | 09:00:00      | 19:00:00       |          9.00 | Present           |
|            10 |          10 | 2026-09-02      | 09:00:00      | 13:00:00       |          4.00 | Half-day          |
+---------------+-------------+-----------------+---------------+----------------+---------------+-------------------+
10 rows in set (0.00 sec)
```

4.2.4 Leave_Request Table

Table Creation (DDL):

CREATE TABLE Leave_Request (

Leave_ID INT PRIMARY KEY,

Employee_ID INT,

```
    Leave_Type VARCHAR(30),
    Start_Date DATE,
    End_Date DATE,
    Number_of_Days INT,
    Reason VARCHAR(100),
    Request_Date DATE,
    Approval_Status VARCHAR(20),
    Approved_By INT,
    FOREIGN KEY (Employee_ID)
        REFERENCES Employee(Employee_ID)
);
```

Data Insertion (DML):

```
INSERT INTO Leave_Request VALUES
(1, 1, 'Casual', '2026-09-05', '2026-09-05', 1,
 'Personal work', '2026-09-01', 'Approved', 10),

(2, 2, 'Sick', '2026-09-06', '2026-09-07', 2,
 'Fever', '2026-09-05', 'Approved', 1),

(3, 3, 'Earned', '2026-09-10', '2026-09-12', 3,
 'Family function', '2026-09-02', 'Pending', NULL),

(4, 4, 'Casual', '2026-09-08', '2026-09-08', 1,
 'Personal work', '2026-09-03', 'Rejected', 1),

(5, 5, 'Sick', '2026-09-15', '2026-09-16', 2,
 'Medical treatment', '2026-09-10', 'Approved', 10),
```

(6, 6, 'Casual', '2026-09-18', '2026-09-18', 1,

'Family work', '2026-09-12', 'Pending', NULL),

(7, 7, 'Earned', '2026-09-20', '2026-09-22', 3,

'Vacation', '2026-09-05', 'Approved', 2),

(8, 8, 'Casual', '2026-09-23', '2026-09-23', 1,

'Personal work', '2026-09-15', 'Rejected', 1),

(9, 9, 'Sick', '2026-09-25', '2026-09-26', 2,

'Health reasons', '2026-09-24', 'Pending', NULL),

(10, 10, 'Earned', '2026-09-28', '2026-09-30', 3,

'Family vacation', '2026-09-10', 'Approved', 1);

Verification Query:

```
SELECT * FROM Leave_Request;
```

Output:

```
mysql> select * from Leave_Request;
+----------+-------------+------------+------------+------------+---------------+-------------------+--------------+-----------------+-------------+
| Leave_ID | Employee_ID | Leave_Type | Start_Date | End_Date   | Number_of_Days | Reason            | Request_Date | Approval_Status | Approved_By |
+----------+-------------+------------+------------+------------+---------------+-------------------+--------------+-----------------+-------------+
|        1 |           1 | Casual     | 2026-09-05 | 2026-09-05 |             1 | Personal work     | 2026-09-01   | Approved        |          10 |
|        2 |           2 | Sick       | 2026-09-06 | 2026-09-07 |             2 | Fever             | 2026-09-05   | Approved        |           1 |
|        3 |           3 | Earned     | 2026-09-10 | 2026-09-12 |             3 | Family function   | 2026-09-02   | Pending         |        NULL |
|        4 |           4 | Casual     | 2026-09-08 | 2026-09-08 |             1 | Personal work     | 2026-09-03   | Rejected        |           1 |
|        5 |           5 | Sick       | 2026-09-15 | 2026-09-16 |             2 | Medical treatment | 2026-09-10   | Approved        |          10 |
|        6 |           6 | Casual     | 2026-09-18 | 2026-09-18 |             1 | Family work       | 2026-09-12   | Pending         |        NULL |
|        7 |           7 | Earned     | 2026-09-20 | 2026-09-22 |             3 | Vacation          | 2026-09-05   | Approved        |           2 |
|        8 |           8 | Casual     | 2026-09-23 | 2026-09-23 |             1 | Personal work     | 2026-09-15   | Rejected        |           1 |
|        9 |           9 | Sick       | 2026-09-25 | 2026-09-26 |             2 | Health reasons    | 2026-09-24   | Pending         |        NULL |
|       10 |          10 | Earned     | 2026-09-28 | 2026-09-30 |             3 | Family vacation   | 2026-09-10   | Approved        |           1 |
+----------+-------------+------------+------------+------------+---------------+-------------------+--------------+-----------------+-------------+
10 rows in set (0.00 sec)
```

4.2.5 Salary Table

Table Creation (DDL):

Salary_ID INT PRIMARY KEY,

```
    Employee_ID INT,
    Basic_Salary DECIMAL(10,2),
    House_Rent_Allowance DECIMAL(10,2),
    Other_Allowances DECIMAL(10,2),
    Deductions DECIMAL(10,2),
    Effective_From DATE,
    Effective_To DATE,
    FOREIGN KEY (Employee_ID)
        REFERENCES Employee(Employee_ID)
);
```

Data Insertion (DML):

```
INSERT INTO Salary VALUES
(1, 1, 50000, 10000, 5000, 3000, '2026-01-01', NULL),
(2, 2, 40000, 8000, 3000, 2000, '2026-01-01', NULL),
(3, 3, 35000, 7000, 2500, 1500, '2026-01-01', NULL),
(4, 4, 30000, 6000, 2000, 1200, '2026-01-01', NULL),
(5, 5, 32000, 6400, 2500, 1400, '2026-01-01', NULL),
(6, 6, 28000, 5600, 1800, 1000, '2026-01-01', NULL),
(7, 7, 45000, 9000, 3500, 2500, '2026-01-01', NULL),
(8, 8, 38000, 7600, 2800, 1800, '2026-01-01', NULL),
(9, 9, 33000, 6600, 2200, 1300, '2026-01-01', NULL),
(10, 10, 42000, 8400, 3000, 2200, '2026-01-01', NULL);
```

Verification Query:

```
SELECT * FROM Salary;
```

Output:

```
mysql> select * from Salary;
+-----------+-------------+--------------+----------------------+------------------+------------+----------------+--------------+
| Salary_ID | Employee_ID | Basic_Salary | House_Rent_Allowance | Other_Allowances | Deductions | Effective_From | Effective_To |
+-----------+-------------+--------------+----------------------+------------------+------------+----------------+--------------+
|         1 |           1 |     50000.00 |             10000.00 |          5000.00 |    3000.00 | 2026-01-01     | NULL         |
|         2 |           2 |     40000.00 |              8000.00 |          3000.00 |    2000.00 | 2026-01-01     | NULL         |
|         3 |           3 |     35000.00 |              7000.00 |          2500.00 |    1500.00 | 2026-01-01     | NULL         |
|         4 |           4 |     30000.00 |              6000.00 |          2000.00 |    1200.00 | 2026-01-01     | NULL         |
|         5 |           5 |     32000.00 |              6400.00 |          2500.00 |    1400.00 | 2026-01-01     | NULL         |
|         6 |           6 |     28000.00 |              5600.00 |          1800.00 |    1000.00 | 2026-01-01     | NULL         |
|         7 |           7 |     45000.00 |              9000.00 |          3500.00 |    2500.00 | 2026-01-01     | NULL         |
|         8 |           8 |     38000.00 |              7600.00 |          2800.00 |    1800.00 | 2026-01-01     | NULL         |
|         9 |           9 |     33000.00 |              6600.00 |          2200.00 |    1300.00 | 2026-01-01     | NULL         |
|        10 |          10 |     42000.00 |              8400.00 |          3000.00 |    2200.00 | 2026-01-01     | NULL         |
+-----------+-------------+--------------+----------------------+------------------+------------+----------------+--------------+
10 rows in set (0.00 sec)
```

4.2.6 Payroll Table

Table Creation (DDL):

```
CREATE TABLE Payroll (
    Payroll_ID INT PRIMARY KEY,
    Employee_ID INT,
    Salary_ID INT,
    Payroll_Month INT,
    Payroll_Year INT,
    Total_Working_Days INT,
    Days_Present INT,
    Days_Absent INT,
    Overtime_Hours DECIMAL(5,2),
    Gross_Salary DECIMAL(10,2),
    Total_Deductions DECIMAL(10,2),
    Net_Salary DECIMAL(10,2),
    Payment_Date DATE,
    Payment_Status VARCHAR(20),
    FOREIGN KEY (Employee_ID)
        REFERENCES Employee(Employee_ID),
    FOREIGN KEY (Salary_ID)
```

```
    REFERENCES Salary(Salary_ID)
);
```

Data Insertion (DML):

```
INSERT INTO Payroll VALUES
(1, 1, 1, 9, 2026, 26, 25, 1, 5, 65000, 5000, 60000, '2026-10-01', 'Paid'),
(2, 2, 2, 9, 2026, 26, 24, 2, 2, 51000, 4000, 47000, '2026-10-01', 'Paid'),
(3, 3, 3, 9, 2026, 26, 23, 3, 0, 44500, 3500, 41000, '2026-10-02', 'Paid'),
(4, 4, 4, 9, 2026, 26, 22, 4, 1, 38000, 3000, 35000, '2026-10-02', 'Pending'),
(5, 5, 5, 9, 2026, 26, 25, 1, 4, 40900, 3400, 37500, '2026-10-03', 'Paid'),
(6, 6, 6, 9, 2026, 26, 24, 2, 0, 35400, 2800, 32600, '2026-10-03', 'Pending'),
(7, 7, 7, 9, 2026, 26, 26, 0, 6, 57500, 4500, 53000, '2026-10-04', 'Paid'),
(8, 8, 8, 9, 2026, 26, 23, 3, 3, 48400, 3800, 44600, '2026-10-04', 'Paid'),
(9, 9, 9, 9, 2026, 26, 25, 1, 2, 41800, 3300, 38500, '2026-10-05', 'Pending'),
(10, 10, 10, 9, 2026, 26, 24, 2, 5, 53400, 4000, 49400, '2026-10-05', 'Paid');
```

Verification Query:

```
SELECT * FROM Payroll;
```

Output:

```
mysql> select * from Payroll;
```

Payroll_ID	Employee_ID	Salary_ID	Payroll_Month	Payroll_Year	Total_Working_Days	Days_Present	Days_Absent	Overtime_Hours	Gross_Salary	Total_Deductions	Net_Salary	Payment_Date	Payment_Status
1	1	1	9	2026	26	25	1	5.00	65000.00	5000.00	60000.00	2026-10-01	Paid
2	2	2	9	2026	26	24	2	2.00	51000.00	4000.00	47000.00	2026-10-01	Paid
3	3	3	9	2026	26	23	3	0.00	44500.00	3500.00	41000.00	2026-10-02	Paid
4	4	4	9	2026	26	22	4	1.00	38000.00	3000.00	35000.00	2026-10-02	Pending
5	5	5	9	2026	26	25	1	4.00	40900.00	3400.00	37500.00	2026-10-03	Paid
6	6	6	9	2026	26	24	2	0.00	35400.00	2800.00	32600.00	2026-10-03	Pending
7	7	7	9	2026	26	26	0	6.00	57500.00	4500.00	53000.00	2026-10-04	Paid
8	8	8	9	2026	26	23	3	3.00	48400.00	3800.00	44600.00	2026-10-04	Paid
9	9	9	9	2026	26	25	1	2.00	41800.00	3300.00	38500.00	2026-10-05	Pending
10	10	10	9	2026	26	24	2	5.00	53400.00	4000.00	49400.00	2026-10-05	Paid

```
10 rows in set (0.00 sec)
```

4.2.7 User_Admin Table

Table Creation (DDL):

```
CREATE TABLE User_Admin (
    User_ID INT PRIMARY KEY,
    Employee_ID INT NULL,
    Username VARCHAR(50) UNIQUE,
```

```
    Password_Hash VARCHAR(100),
    Role VARCHAR(30),
    Created_At DATE,
    Account_Status VARCHAR(20),
    FOREIGN KEY (Employee_ID)
        REFERENCES Employee(Employee_ID)
);
```

Data Insertion (DML):

INSERT INTO User_Admin VALUES

(1, 1, 'rahul_admin', 'hash101', 'Admin', '2026-01-01', 'Active'),

(2, 2, 'priya_user', 'hash102', 'Employee', '2026-01-02', 'Active'),

(3, 3, 'amit_user', 'hash103', 'Employee', '2026-01-03', 'Active'),

(4, 4, 'sneha_user', 'hash104', 'Employee', '2026-01-04', 'Active'),

(5, 5, 'kiran_user', 'hash105', 'Employee', '2026-01-05', 'Active'),

(6, 6, 'ananya_user', 'hash106', 'Manager', '2026-01-06', 'Active'),

(7, 7, 'vikram_user', 'hash107', 'Employee', '2026-01-07', 'Active'),

(8, 8, 'divya_user', 'hash108', 'Employee', '2026-01-08', 'Inactive'),

(9, 9, 'arjun_user', 'hash109', 'Employee', '2026-01-09', 'Active'),

(10, NULL, 'external_admin', 'hash110', 'Admin', '2026-01-10', 'Active');

Verification Query:

```
SELECT * FROM User_Admin;
```

Output:

```
mysql> SELECT * FROM User_Admin;
+---------+-------------+----------------+---------------+----------+------------+----------------+
| User_ID | Employee_ID | Username       | Password_Hash | Role     | Created_At | Account_Status |
+---------+-------------+----------------+---------------+----------+------------+----------------+
|       1 |           1 | rahul_admin    | hash101       | Admin    | 2026-01-01 | Active         |
|       2 |           2 | priya_user     | hash102       | Employee | 2026-01-02 | Active         |
|       3 |           3 | amit_user      | hash103       | Employee | 2026-01-03 | Active         |
|       4 |           4 | sneha_user     | hash104       | Employee | 2026-01-04 | Active         |
|       5 |           5 | kiran_user     | hash105       | Employee | 2026-01-05 | Active         |
|       6 |           6 | ananya_user    | hash106       | Manager  | 2026-01-06 | Active         |
|       7 |           7 | vikram_user    | hash107       | Employee | 2026-01-07 | Active         |
|       8 |           8 | divya_user     | hash108       | Employee | 2026-01-08 | Inactive       |
|       9 |           9 | arjun_user     | hash109       | Employee | 2026-01-09 | Active         |
|      10 |        NULL | external_admin | hash110       | Admin    | 2026-01-10 | Active         |
+---------+-------------+----------------+---------------+----------+------------+----------------+
10 rows in set (0.00 sec)
```

5. SQL QUERIES AND OUTPUTS

5.1 SQL Queries with Outputs

5.1.1 ANY, ALL, UNION and INTERSECT

5.1.1.1 Query 1: Display employees whose basic salary is greater than any basic salary of employees in department 2.

SQL Query:

```
SELECT e.Employee_ID,
      e.First_Name,
      e.Last_Name,
      s.Basic_Salary
FROM Employee e
JOIN Salary s
   ON e.Employee_ID = s.Employee_ID
WHERE s.Basic_Salary > ANY (
   SELECT s2.Basic_Salary
   FROM Salary s2
   JOIN Employee e2
      ON s2.Employee_ID = e2.Employee_ID
   WHERE e2.Department_ID = 2
);
```

OUTPUT:

Employee_ID	First_Name	Last_Name	Basic_Salary
1	Rahul	Sharma	50000.00
7	Vikram	Singh	45000.00
10	Meena	Das	42000.00

3 rows in set (0.00 sec)

5.1.1.2 Query 2: Display employees whose basic salary is greater than all basic salaries of employees in department 3.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      e.Last_Name,
      s.Basic_Salary
FROM Employee e
JOIN Salary s
   ON e.Employee_ID = s.Employee_ID
WHERE s.Basic_Salary > ALL (
   SELECT s2.Basic_Salary
   FROM Salary s2
   JOIN Employee e2
      ON s2.Employee_ID = e2.Employee_ID
   WHERE e2.Department_ID = 3
);
```

OUTPUT:

Employee_ID	First_Name	Last_Name	Basic_Salary
1	Rahul	Sharma	50000.00
2	Priya	Reddy	40000.00
7	Vikram	Singh	45000.00
8	Divya	Mohan	38000.00
10	Meena	Das	42000.00

5 rows in set (0.00 sec)

5.1.1.3 Query 3: Display employee names belonging to either the HR or Finance department using UNION.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      e.Last_Name
FROM Employee e
JOIN Department d
   ON e.Department_ID = d.Department_ID
WHERE d.Department_Name = 'HR'

UNION

SELECT e.Employee_ID,
      e.First_Name,
      e.Last_Name
FROM Employee e
JOIN Department d
   ON e.Department_ID = d.Department_ID
WHERE d.Department_Name = 'Finance';
```

OUTPUT:

```
+-------------+------------+-----------+
| Employee_ID | First_Name | Last_Name |
+-------------+------------+-----------+
|           1 | Rahul      | Sharma    |
|           3 | Amit       | Kumar     |
+-------------+------------+-----------+
2 rows in set (0.00 sec)
```

5.1.1.4 Query 4: Find employee IDs that appear in both the Attendance and Leave_Request tables using an appropriate MySQL query.

SQL QUERY:

```
SELECT DISTINCT a.Employee_ID
FROM Attendance a
INNER JOIN Leave_Request l
   ON a.Employee_ID = l.Employee_ID;
```

OUTPUT:

```
+-------------+
| Employee_ID |
+-------------+
|           1 |
|           2 |
|           3 |
|           4 |
|           5 |
|           6 |
|           7 |
|           8 |
|           9 |
|          10 |
+-------------+
10 rows in set (0.00 sec)
```

5.1.1.5 Query 5: Display employees whose basic salary exceeds ₹35,000 and whose employment status is Active.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      e.Last_Name,
      s.Basic_Salary
FROM Employee e
JOIN Salary s
   ON e.Employee_ID = s.Employee_ID
WHERE s.Basic_Salary > 35000
  AND e.Employment_Status = 'Active';
```

OUTPUT:

```
+-------------+------------+-----------+--------------+
| Employee_ID | First_Name | Last_Name | Basic_Salary |
+-------------+------------+-----------+--------------+
|           1 | Rahul      | Sharma    |     50000.00 |
|           2 | Priya      | Reddy     |     40000.00 |
|           7 | Vikram     | Singh     |     45000.00 |
|           8 | Divya      | Mohan     |     38000.00 |
|          10 | Meena      | Das       |     42000.00 |
+-------------+------------+-----------+--------------+
5 rows in set (0.00 sec)
```

5.1.2 Nested Subqueries

5.1.2.1 Query 1: Display employees earning more than the average basic salary.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      e.Last_Name,
      s.Basic_Salary
FROM Employee e
JOIN Salary s
   ON e.Employee_ID = s.Employee_ID
WHERE s.Basic_Salary > (
   SELECT AVG(Basic_Salary)
   FROM Salary
);
```

OUTPUT:

```
+-------------+------------+-----------+--------------+
| Employee_ID | First_Name | Last_Name | Basic_Salary |
+-------------+------------+-----------+--------------+
|           1 | Rahul      | Sharma    |     50000.00 |
|           2 | Priya      | Reddy     |     40000.00 |
|           7 | Vikram     | Singh     |     45000.00 |
|           8 | Divya      | Mohan     |     38000.00 |
|          10 | Meena      | Das       |     42000.00 |
+-------------+------------+-----------+--------------+
5 rows in set (0.00 sec)
```

5.1.2.2 Query 2: Display employees receiving the highest basic salary.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      e.Last_Name,
      s.Basic_Salary
FROM Employee e
JOIN Salary s
   ON e.Employee_ID = s.Employee_ID
WHERE s.Basic_Salary = (
   SELECT MAX(Basic_Salary)
   FROM Salary
);
```

OUTPUT:

```
+-------------+------------+-----------+--------------+
| Employee_ID | First_Name | Last_Name | Basic_Salary |
+-------------+------------+-----------+--------------+
|           1 | Rahul      | Sharma    |     50000.00 |
+-------------+------------+-----------+--------------+
1 row in set (0.00 sec)
```

5.1.2.3 Query 3: Display employees earning less than the average basic salary.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      e.Last_Name,
      s.Basic_Salary
FROM Employee e
JOIN Salary s
   ON e.Employee_ID = s.Employee_ID
WHERE s.Basic_Salary < (
   SELECT AVG(Basic_Salary)
   FROM Salary
);
```

OUTPUT:

```
+-------------+------------+-----------+--------------+
| Employee_ID | First_Name | Last_Name | Basic_Salary |
+-------------+------------+-----------+--------------+
|           3 | Amit       | Kumar     |     35000.00 |
|           4 | Sneha      | Rao       |     30000.00 |
|           5 | Kiran      | Reddy     |     32000.00 |
|           6 | Ananya     | Patel     |     28000.00 |
|           9 | Arjun      | Verma     |     33000.00 |
+-------------+------------+-----------+--------------+
5 rows in set (0.01 sec)
```

5.1.2.4 Query 4: Display employees belonging to the department with the highest average basic salary.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
```

```
    e.Last_Name,
    e.Department_ID
FROM Employee e
WHERE e.Department_ID IN (
  SELECT e2.Department_ID
  FROM Employee e2
  JOIN Salary s2
    ON e2.Employee_ID = s2.Employee_ID
  GROUP BY e2.Department_ID
  HAVING AVG(s2.Basic_Salary) >= ALL (
    SELECT AVG(s3.Basic_Salary)
    FROM Employee e3
    JOIN Salary s3
      ON e3.Employee_ID = s3.Employee_ID
    GROUP BY e3.Department_ID
  )
);
```

OUTPUT:

```
+-------------+------------+-----------+---------------+
| Employee_ID | First_Name | Last_Name | Department_ID |
+-------------+------------+-----------+---------------+
|           1 | Rahul      | Sharma    |             1 |
+-------------+------------+-----------+---------------+
1 row in set (0.00 sec)
```

5.1.3 Correlated Subqueries

5.1.3.1 Query1: Display employees earning more than the average salary of their own department.

SQL QUERY:

```
SELECT e.Employee_ID,
    e.First_Name,
    e.Last_Name,
    e.Department_ID,
    s.Basic_Salary
FROM Employee e
JOIN Salary s
  ON e.Employee_ID = s.Employee_ID
WHERE s.Basic_Salary > (
  SELECT AVG(s2.Basic_Salary)
```

```
   FROM Employee e2
   JOIN Salary s2
      ON e2.Employee_ID = s2.Employee_ID
   WHERE e2.Department_ID = e.Department_ID
);
```

OUTPUT:
EMPTY SET (0.00 sec)

5.1.3.2 Query 2: Display employees who have submitted at least one leave request.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      e.Last_Name
FROM Employee e
WHERE EXISTS (
   SELECT 1
   FROM Leave_Request l
   WHERE l.Employee_ID = e.Employee_ID
);
```

OUTPUT:

Employee_ID	First_Name	Last_Name
1	Rahul	Sharma
2	Priya	Reddy
3	Amit	Kumar
4	Sneha	Rao
5	Kiran	Reddy
6	Ananya	Patel
7	Vikram	Singh
8	Divya	Mohan
9	Arjun	Verma
10	Meena	Das

10 rows in set (0.00 sec)

5.1.3.3 Query 3: Display employees who have never submitted a leave request.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      e.Last_Name
FROM Employee e
WHERE NOT EXISTS (
   SELECT 1
   FROM Leave_Request l
   WHERE l.Employee_ID = e.Employee_ID
);
```

OUTPUT:
EMPTY SET(0.00 sec)

5.1.3.4 Query 4: Display departments that have at least one employee.

SQL QUERY:

```
SELECT d.Department_ID,
      d.Department_Name
FROM Department d
WHERE EXISTS (
   SELECT 1
   FROM Employee e
   WHERE e.Department_ID = d.Department_ID
);
```

OUTPUT:

```
+---------------+-----------------+
| Department_ID | Department_Name |
+---------------+-----------------+
|             1 | HR              |
|             2 | IT              |
|             3 | Finance         |
|             4 | Marketing       |
|             5 | Sales           |
|             6 | Operations      |
|             7 | Development     |
|             8 | Testing         |
|             9 | Support         |
|            10 | Administration  |
+---------------+-----------------+
10 rows in set (0.00 sec)
```

5.1.4 Aggregate Functions

5.1.4.1 Query 1: Find the total number of employees.

SQL QUERY:

```
SELECT COUNT(*) AS Total_Employees
FROM Employee;
```

OUTPUT:

```
+-----------------+
| Total_Employees |
+-----------------+
|              10 |
+-----------------+
1 row in set (0.00 sec)
```

5.1.4.2 Query 2: Find the maximum and minimum basic salaries.

SQL QUERY:

```
SELECT MAX(Basic_Salary) AS Highest_Salary,
    MIN(Basic_Salary) AS Lowest_Salary
FROM Salary;
```

OUTPUT:

```
+----------------+---------------+
| Highest_Salary | Lowest_Salary |
+----------------+---------------+
|       50000.00 |      28000.00 |
+----------------+---------------+
1 row in set (0.01 sec)
```

5.1.4.3 Query 3: Calculate the average basic salary.

SQL QUERY:

```
SELECT AVG(Basic_Salary) AS Average_Salary
FROM Salary;
```

OUTPUT:

```
+----------------+
| Average_Salary |
+----------------+
|   37300.000000 |
+----------------+
1 row in set (0.00 sec)
```

5.1.4.4 Query 4: Calculate the total basic salary of all employees.

SQL QUERY:

```
SELECT SUM(Basic_Salary) AS Total_Basic_Salary
FROM Salary;
```

OUTPUT:

```
+--------------------+
| Total_Basic_Salary |
+--------------------+
|          373000.00 |
+--------------------+
1 row in set (0.00 sec)
```

5.1.4.5 Query 5: Count employees whose employment status is Active.

SQL QUERY:

```
SELECT COUNT(*) AS Active_Employees
FROM Employee
WHERE Employment_Status = 'Active';
```

OUTPUT:

```
+------------------+
| Active_Employees |
+------------------+
|               10 |
+------------------+
1 row in set (0.01 sec)
```

5.1.5 GROUP BY and HAVING

5.1.5.1 Query 1: Count the employees in each department.

SQL QUERY:

```
SELECT d.Department_ID,
      d.Department_Name,
      COUNT(e.Employee_ID) AS Total_Employees
FROM Department d
LEFT JOIN Employee e
   ON d.Department_ID = e.Department_ID
GROUP BY d.Department_ID, d.Department_Name;
```

OUTPUT:

```
+---------------+-----------------+-----------------+
| Department_ID | Department_Name | Total_Employees |
+---------------+-----------------+-----------------+
|             1 | HR              |               1 |
|             2 | IT              |               1 |
|             3 | Finance         |               1 |
|             4 | Marketing       |               1 |
|             5 | Sales           |               1 |
|             6 | Operations      |               1 |
|             7 | Development     |               1 |
|             8 | Testing         |               1 |
|             9 | Support         |               1 |
|            10 | Administration  |               1 |
+---------------+-----------------+-----------------+
10 rows in set (0.00 sec)
```

5.1.5.2 Calculate the average basic salary for each department.

SQL QUERY:

```
SELECT e.Department_ID,
      d.Department_Name,
      AVG(s.Basic_Salary) AS Average_Salary
FROM Employee e
JOIN Department d
   ON e.Department_ID = d.Department_ID
JOIN Salary s
   ON e.Employee_ID = s.Employee_ID
GROUP BY e.Department_ID, d.Department_Name;
```

OUTPUT:

Department_ID	Department_Name	Average_Salary
1	HR	50000.000000
2	IT	40000.000000
3	Finance	35000.000000
4	Marketing	30000.000000
5	Sales	32000.000000
6	Operations	28000.000000
7	Development	45000.000000
8	Testing	38000.000000
9	Support	33000.000000
10	Administration	42000.000000

10 rows in set (0.00 sec)

5.1.5.3 Query 3: Display departments having more than one employee.

SQL QUERY:

```
SELECT d.Department_ID,
     d.Department_Name,
     COUNT(e.Employee_ID) AS Total_Employees
FROM Department d
JOIN Employee e
   ON d.Department_ID = e.Department_ID
GROUP BY d.Department_ID, d.Department_Name
HAVING COUNT(e.Employee_ID) > 1;
```

OUTPUT:
Empty Set(0.00 sec)

5.1.5.4 Query 4: Display departments whose average basic salary exceeds ₹35,000.

SQL QUERY:

```
SELECT d.Department_ID,
     d.Department_Name,
     AVG(s.Basic_Salary) AS Average_Salary
FROM Department d
JOIN Employee e
   ON d.Department_ID = e.Department_ID
JOIN Salary s
   ON e.Employee_ID = s.Employee_ID
GROUP BY d.Department_ID, d.Department_Name
```

HAVING AVG(s.Basic_Salary) > 35000;

OUTPUT:

Department_ID	Department_Name	Average_Salary
1	HR	50000.000000
2	IT	40000.000000
7	Development	45000.000000
8	Testing	38000.000000
10	Administration	42000.000000

5 rows in set (0.00 sec)

5.1.5.5 Query 5: Count employees according to employment status.

SQL QUERY:

```
SELECT Employment_Status,
    COUNT(*) AS Total_Employees
FROM Employee
GROUP BY Employment_Status;
```

OUTPUT:

Employment_Status	Total_Employees
Active	10

1 row in set (0.00 sec)

5.1.6 Views

5.1.6.1 Query 1: Create a view displaying employee IDs, names and basic salaries.

SQL QUERY:

TABLE:

```
CREATE VIEW Employee_Salary_View AS
SELECT e.Employee_ID,
    CONCAT(e.First_Name, ' ', e.Last_Name) AS Employee_Name,
    s.Basic_Salary
```

FROM Employee e
JOIN Salary s
ON e.Employee_ID = s.Employee_ID;

DISPLAY:
SELECT * FROM Employee_Salary_View;

OUTPUT:

Employee_ID	Employee_Name	Basic_Salary
1	Rahul Sharma	50000.00
2	Priya Reddy	40000.00
3	Amit Kumar	35000.00
4	Sneha Rao	30000.00
5	Kiran Reddy	32000.00
6	Ananya Patel	28000.00
7	Vikram Singh	45000.00
8	Divya Mohan	38000.00
9	Arjun Verma	33000.00
10	Meena Das	42000.00

10 rows in set (0.00 sec)

5.1.6.2 Query 2: Create a view displaying department names and employee names.

SQL QUERY:

TABLE:
CREATE VIEW Department_Employee_View AS
SELECT d.Department_Name,
e.Employee_ID,
e.First_Name,
e.Last_Name
FROM Department d
JOIN Employee e
ON d.Department_ID = e.Department_ID;

DISPLAY:
SELECT * FROM Department_Employee_View;

OUTPUT:

Department_Name	Employee_ID	First_Name	Last_Name
HR	1	Rahul	Sharma
IT	2	Priya	Reddy
Finance	3	Amit	Kumar
Marketing	4	Sneha	Rao
Sales	5	Kiran	Reddy
Operations	6	Ananya	Patel
Development	7	Vikram	Singh
Testing	8	Divya	Mohan
Support	9	Arjun	Verma
Administration	10	Meena	Das

10 rows in set (0.00 sec)

5.1.6.3 Query 3: Create a view showing employees whose basic salary exceeds ₹35,000.

SQL QUERY:

TABLE:

```
CREATE VIEW High_Salary_Employees AS
SELECT e.Employee_ID,
      e.First_Name,
      e.Last_Name,
      s.Basic_Salary
FROM Employee e
JOIN Salary s
   ON e.Employee_ID = s.Employee_ID
WHERE s.Basic_Salary > 35000;
```

DISPLAY:

```
SELECT * FROM High_Salary_Employees;
```

OUTPUT:

```
+-------------+------------+-----------+--------------+
| Employee_ID | First_Name | Last_Name | Basic_Salary |
+-------------+------------+-----------+--------------+
|           1 | Rahul      | Sharma    |     50000.00 |
|           2 | Priya      | Reddy     |     40000.00 |
|           7 | Vikram     | Singh     |     45000.00 |
|           8 | Divya      | Mohan     |     38000.00 |
|          10 | Meena      | Das       |     42000.00 |
+-------------+------------+-----------+--------------+
5 rows in set (0.00 sec)
```

5.1.6.4 Query 4: Drop the Employee_Salary_View.

SQL QUERY:

```
DROP VIEW Employee_Salary_View;
```

OUTPUT:

```
mysql> DROP VIEW Employee_Salary_View;
Query OK, 0 rows affected (0.06 sec)
```

5.1.7 INSERT Triggers

5.1.7.1 Query 1: Prevent negative working hours when inserting attendance.

SQL QUERY:

TRIGGER:

```
DELIMITER //

CREATE TRIGGER Check_Working_Hours
BEFORE INSERT ON Attendance
FOR EACH ROW
BEGIN
   IF NEW.Working_Hours < 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Working hours cannot be negative';
   END IF;
END //
```

DELIMITER ;

TEST:

```
INSERT INTO Attendance
VALUES (11, 1, '2026-09-03',
      '09:00:00', '18:00:00', -5, 'Present');
```

OUTPUT:

ERROR 1644 (45000): Working hours cannot be negative

5.1.7.2 Query 2: Record inserted attendance entries in a log table.

SQL QUERY:

TABLE:

```
CREATE TABLE Attendance_Log (
   Log_ID INT AUTO_INCREMENT PRIMARY KEY,
   Attendance_ID INT,
   Employee_ID INT,
   Attendance_Date DATE,
   Action_Type VARCHAR(20),
   Log_Time DATETIME
);
```

TRIGGER:

```
DELIMITER //

CREATE TRIGGER Log_Attendance_Insert
FOR EACH ROW
BEGIN
   INSERT INTO Attendance_Log
      (Attendance_ID, Employee_ID, Attendance_Date,
       Action_Type, Log_Time)
   VALUES
      (NEW.Attendance_ID, NEW.Employee_ID,
       NEW.Attendance_Date, 'INSERT', NOW());
END //

DELIMITER ;
```

TEST:

```
INSERT INTO Attendance
VALUES (11, 1, '2026-09-03',
      '09:00:00', '18:00:00', 8, 'Present');
```

DISPLAY:

```
SELECT * FROM Attendance_Log;
```

OUTPUT:

```
---------+---------------+-------------+-----------------+-------------+---------------------+
 Log_ID | Attendance_ID | Employee_ID | Attendance_Date | Action_Type | Log_Time            |
---------+---------------+-------------+-----------------+-------------+---------------------+
      1 |            11 |           1 | 2026-09-03      | INSERT      | 2026-10-11 22:39:19 |
---------+---------------+-------------+-----------------+-------------+---------------------+
1 row in set (0.00 sec)
```

5.1.7.3 Query 3: Prevent negative basic salaries during insertion.

SQL QUERY:

TRIGGER:

```
DELIMITER //

CREATE TRIGGER Check_Salary_Insert
BEFORE INSERT ON Salary
FOR EACH ROW
BEGIN
   IF NEW.Basic_Salary < 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Basic salary cannot be negative';
   END IF;
END //

DELIMITER ;
```

TEST:

```
INSERT INTO Salary
VALUES (11, 1, -1000, 0, 0, 0, '2026-10-01', NULL);
```

OUTPUT:

ERROR 1644 (45000): Basic salary cannot be negative

5.1.8 UPDATE Triggers

5.1.8.1 QUERY 1: Prevent updating basic salary to a negative amount.

SQL QUERY:

TRIGGER:

```
DELIMITER //

CREATE TRIGGER Check_Salary_Update
BEFORE UPDATE ON Salary
FOR EACH ROW
BEGIN
   IF NEW.Basic_Salary < 0 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Basic salary cannot be negative';
   END IF;
END //

DELIMITER ;
```

TEST:

```
UPDATE Salary
SET Basic_Salary = -5000
WHERE Salary_ID = 1;
```

OUTPUT:

ERROR 1644 (45000): Basic salary cannot be negative

5.1.8.2 Query 2: Record payroll payment-status changes.

SQL QUERY:

TABLE:

```
CREATE TABLE Payroll_Status_Log (
   Log_ID INT AUTO_INCREMENT PRIMARY KEY,
   Payroll_ID INT,
   Old_Status VARCHAR(20),
   New_Status VARCHAR(20),
   Change_Time DATETIME
);
```

TRIGGER:

```
DELIMITER //

CREATE TRIGGER Log_Payroll_Status
AFTER UPDATE ON Payroll
FOR EACH ROW
BEGIN
   IF NOT (OLD.Payment_Status <=> NEW.Payment_Status) THEN
      INSERT INTO Payroll_Status_Log
         (Payroll_ID, Old_Status, New_Status, Change_Time)
      VALUES
         (NEW.Payroll_ID, OLD.Payment_Status,
          NEW.Payment_Status, NOW());
   END IF;
END //

DELIMITER ;
```

TEST:

```
UPDATE Payroll
SET Payment_Status = 'Paid'
WHERE Payroll_ID = 4;
```

DISPLAY:

```
SELECT * FROM Payroll_Status_Log;
```

OUTPUT:

```
+--------+------------+------------+------------+---------------------+
| Log_ID | Payroll_ID | Old_Status | New_Status | Change_Time         |
+--------+------------+------------+------------+---------------------+
|      1 |          4 | Pending    | Paid       | 2026-10-11 22:42:08 |
+--------+------------+------------+------------+---------------------+
1 row in set (0.00 sec)
```

5.1.8.3 Query 3: Prevent an employee's email from being changed to an empty string.

SQL QUERY:

TRIGGER:

DELIMITER //

CREATE TRIGGER Check_Employee_Email
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
IF NEW.Email IS NULL OR TRIM(NEW.Email) = '' THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Email cannot be empty';
END IF;
END //

DELIMITER ;

TEST:
UPDATE Employee
SET Email = ''
WHERE Employee_ID = 1;

OUTPUT:
ERROR 1644 (45000) : Email cannot be empty

5.1.9 DELETE Triggers

5.1.9.1 Query 1: Record deleted leave requests in a log table.

SQL QUERY:

TABLE:

```
CREATE TABLE Leave_Delete_Log (
   Log_ID INT AUTO_INCREMENT PRIMARY KEY,
   Leave_ID INT,
   Employee_ID INT,
   Leave_Type VARCHAR(30),
   Deleted_At DATETIME
);
```

TRIGGER:

```
DELIMITER //

CREATE TRIGGER Log_Leave_Delete
AFTER DELETE ON Leave_Request
```

```
FOR EACH ROW
BEGIN
   INSERT INTO Leave_Delete_Log
      (Leave_ID, Employee_ID, Leave_Type, Deleted_At)
   VALUES
      (OLD.Leave_ID, OLD.Employee_ID,
       OLD.Leave_Type, NOW());
END //

DELIMITER ;
```

TEST:

```
DELETE FROM Leave_Request
WHERE Leave_ID = 4;
```

DISPLAY:

```
SELECT * FROM Leave_Delete_Log;
```

OUTPUT:

```
+--------+----------+-------------+------------+---------------------+
| Log_ID | Leave_ID | Employee_ID | Leave_Type | Deleted_At          |
+--------+----------+-------------+------------+---------------------+
|      1 |        4 |           4 | Casual     | 2026-10-11 22:43:51 |
+--------+----------+-------------+------------+---------------------+
l row in set (0.00 sec)
```

5.1.9.2 Query 2: Record deleted attendance entries.

SQL QUERY:

TABLE:

```
CREATE TABLE Attendance_Delete_Log (
   Log_ID INT AUTO_INCREMENT PRIMARY KEY,
   Attendance_ID INT,
   Employee_ID INT,
   Attendance_Date DATE,
   Deleted_At DATETIME
);
```

TRIGGER:

```
DELIMITER //

CREATE TRIGGER Log_Attendance_Delete
AFTER DELETE ON Attendance
FOR EACH ROW
BEGIN
   INSERT INTO Attendance_Delete_Log
      (Attendance_ID, Employee_ID, Attendance_Date, Deleted_At)
   VALUES
      (OLD.Attendance_ID, OLD.Employee_ID,
       OLD.Attendance_Date, NOW());
END //

DELIMITER ;
```

TEST:

```
DELETE FROM Attendance
WHERE Attendance_ID = 10;
```

DISPLAY:

```
SELECT * FROM Attendance_Delete_Log;
```

OUTPUT:

```
+--------+---------------+-------------+-----------------+---------------------+
| Log_ID | Attendance_ID | Employee_ID | Attendance_Date | Deleted_At          |
+--------+---------------+-------------+-----------------+---------------------+
|      1 |            10 |          10 | 2026-09-02      | 2026-10-11 22:44:39 |
+--------+---------------+-------------+-----------------+---------------------+
1 row in set (0.00 sec)
```

5.1.10 Stored Procedures

5.1.10.1 Query 1: Display all employees.

SQL QUERY:

PROCEDURE:

```
DELIMITER //
```

```
CREATE PROCEDURE Get_All_Employees()
BEGIN
   SELECT * FROM Employee;
END //

DELIMITER ;
```

CALL:

```
CALL Get_All_Employees();
```

OUTPUT:

Employee_ID	First_Name	Last_Name	Gender	Date_of_Birth	Email	Phone_Number	Address	Department_ID	Designation	Date_of_Joining	Employment_Status
1	Rahul	Sharma	Male	1995-05-10	rahul@gmail.com	9876500001	Hyderabad	1	HR Manager	2020-01-10	Active
2	Priya	Reddy	Female	1998-03-15	priya@gmail.com	9876500002	Hyderabad	2	Software Engineer	2021-06-15	Active
3	Amit	Kumar	Male	1996-07-20	amit@gmail.com	9876500003	Vijayawada	3	Accountant	2022-02-01	Active
4	Sneha	Rao	Female	1997-09-12	sneha@gmail.com	9876500004	Warangal	4	Marketing Executive	2021-08-10	Active
5	Kiran	Reddy	Male	1994-11-25	kiran@gmail.com	9876500005	Guntur	5	Sales Executive	2019-04-05	Active
6	Ananya	Patel	Female	1999-01-18	ananya@gmail.com	9876500006	Hyderabad	6	Operations Executive	2023-01-12	Active
7	Vikram	Singh	Male	1995-06-30	vikram@gmail.com	9876500007	Nizamabad	7	Developer	2020-09-20	Active
8	Divya	Mohan	Female	1998-12-05	divya@gmail.com	9876500008	Karimnagar	8	Test Engineer	2022-07-01	Active
9	Arjun	Verma	Male	1993-04-22	arjun@gmail.com	9876500009	Hyderabad	9	Support Engineer	2018-03-15	Active
10	Meena	Das	Female	1997-08-14	meena@gmail.com	9876500010	Khammam	10	Administrator	2021-11-01	Active

```
10 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

5.1.10.2 Query 2: Display an employee using Employee_ID.

SQL QUERY:

PROCEDURE:

```
DELIMITER //

CREATE PROCEDURE Get_Employee_By_ID(IN emp_id INT)
BEGIN
   SELECT *
   FROM Employee
   WHERE Employee_ID = emp_id;
END //

DELIMITER ;
```

CALL:

```
CALL Get_Employee_By_ID(2);
```

OUTPUTL:

```
mysql> CALL Get_Employee_By_ID(2);
+-------------+------------+-----------+--------+---------------+-----------------+--------------+----------+---------------+--------------------+-----------------+-------------------+
| Employee_ID | First_Name | Last_Name | Gender | Date_of_Birth | Email           | Phone_Number | Address  | Department_ID | Designation        | Date_of_Joining | Employment_Status |
+-------------+------------+-----------+--------+---------------+-----------------+--------------+----------+---------------+--------------------+-----------------+-------------------+
|           2 | Priya      | Reddy     | Female | 1998-03-15    | priya@gmail.com | 9876500002   | Hyderabad |             2 | Software Engineer | 2021-06-15      | Active            |
+-------------+------------+-----------+--------+---------------+-----------------+--------------+----------+---------------+--------------------+-----------------+-------------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

5.1.10.3 Query 3: Display employees belonging to a department.

SQL QUERY:

PROCEDURE:

```
DELIMITER //

CREATE PROCEDURE Get_Employees_By_Department(IN dept_id INT)
BEGIN
   SELECT Employee_ID, First_Name, Last_Name, Designation
   FROM Employee
   WHERE Department_ID = dept_id;
END //

DELIMITER ;
```

CALL:

```
CALL Get_Employees_By_Department(2);
```

OUTPUT:

```
+-------------+------------+-----------+--------------------+
| Employee_ID | First_Name | Last_Name | Designation        |
+-------------+------------+-----------+--------------------+
|           2 | Priya      | Reddy     | Software Engineer |
+-------------+------------+-----------+--------------------+
1 row in set (0.00 sec)
```

5.10.4 Query 4: Calculate the gross salary of an employee.

SQL QUERY:

PROCEDURE:

```
DELIMITER //
```

```
CREATE PROCEDURE Get_Gross_Salary(IN emp_id INT)
BEGIN
   SELECT Employee_ID,
        Basic_Salary,
        House_Rent_Allowance,
        Other_Allowances,
        Basic_Salary + House_Rent_Allowance
          + Other_Allowances AS Gross_Salary
   FROM Salary
   WHERE Employee_ID = emp_id;
END //

DELIMITER ;
```

CALL:

```
CALL Get_Gross_Salary(1);
```

OUTPUT:

```
+-------------+--------------+----------------------+------------------+--------------+
| Employee_ID | Basic_Salary | House_Rent_Allowance | Other_Allowances | Gross_Salary |
+-------------+--------------+----------------------+------------------+--------------+
|           1 |     50000.00 |             10000.00 |          5000.00 |     65000.00 |
+-------------+--------------+----------------------+------------------+--------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

5.1.10.5 Query 5: Display payroll for a given month and year.

SQL QUERY:

PROCEDURE:

```
DELIMITER //

CREATE PROCEDURE Get_Monthly_Payroll(
   IN p_month INT,
   IN p_year INT
)
BEGIN
   SELECT *
   FROM Payroll
   WHERE Payroll_Month = p_month
     AND Payroll_Year = p_year;
```

END //

DELIMITER ;

CALL:

CALL Get_Monthly_Payroll(9, 2026);

OUTPUT:

Payroll_ID	Employee_ID	Salary_ID	Payroll_Month	Payroll_Year	Total_Working_Days	Days_Present	Days_Absent	Overtime_Hours	Gross_Salary	Total_Deductions	Net_Salary	Payment_Date	Payment_Status
1	1	1	9	2026	26	25	1	5.00	65000.00	5000.00	60000.00	2026-10-01	Paid
2	2	2	9	2026	26	24	2	2.00	51000.00	4000.00	47000.00	2026-10-01	Paid
3	3	3	9	2026	26	23	3	0.00	44500.00	3500.00	41000.00	2026-10-02	Paid
4	4	4	9	2026	26	22	4	1.00	38000.00	3000.00	35000.00	2026-10-02	Paid
5	5	5	9	2026	26	25	1	4.00	40900.00	3400.00	37500.00	2026-10-03	Paid
6	6	6	9	2026	26	24	2	0.00	35400.00	2800.00	32600.00	2026-10-03	Pending
7	7	7	9	2026	26	26	0	6.00	57500.00	4500.00	53000.00	2026-10-04	Paid
8	8	8	9	2026	26	23	3	3.00	48400.00	3800.00	44600.00	2026-10-04	Paid
9	9	9	9	2026	26	25	1	2.00	41800.00	3300.00	38500.00	2026-10-05	Pending
10	10	10	9	2026	26	24	2	5.00	53400.00	4000.00	49400.00	2026-10-05	Paid

10 rows in set (0.00 sec)

5.1.11 IN,OUT,andINOUT Parameters

5.1.11.1 Query 1: N parameter: Retrieve employee details.

SQL QUERY:
PRECEDURE:

DELIMITER //

CREATE PROCEDURE Employee_Details(IN emp_id INT)
BEGIN
  SELECT *
  FROM Employee
  WHERE Employee_ID = emp_id;
END //

DELIMITER ;

CALL:

CALL Employee_Details(5);

OUTPUT:

Employee_ID	First_Name	Last_Name	Gender	Date_of_Birth	Email	Phone_Number	Address	Department_ID	Designation	Date_of_Joining	Employment_Status
5	Kiran	Reddy	Male	1994-11-25	kiran@gmail.com	9876500005	Guntur	5	Sales Executive	2019-04-05	Active

1 row in set (0.00 sec)

5.1.11.2 Query 2: OUT parameter: Return the total number of employees.

SQL QUERY:

PROCEDURE:

```
DELIMITER //

CREATE PROCEDURE Count_Employees(OUT total INT)
BEGIN
   SELECT COUNT(*)
   INTO total
   FROM Employee;
END //

DELIMITER ;
```

CALL:

```
CALL Count_Employees(@total);
```

DISPLAY:

```
SELECT @total AS Total_Employees;
```

OUTPUT:

```
+-----------------+
| Total_Employees |
+-----------------+
|              10 |
+-----------------+
1 row in set (0.00 sec)
```

5.1.11.3 Query 3: OUT parameter: Return the average basic salary.

SQL QUERY:

PROCEDURE:

```
DELIMITER //

CREATE PROCEDURE Average_Basic_Salary(OUT avg_salary DECIMAL(10,2))
BEGIN
   SELECT AVG(Basic_Salary)
   INTO avg_salary
   FROM Salary;
```

END //

DELIMITER ;

CALL:

CALL Average_Basic_Salary(@avg_salary);

DISPLAYE:

SELECT @avg_salary AS Average_Basic_Salary;

OUTPUT:

```
+----------------------+
| Average_Basic_Salary |
+----------------------+
|             37300.00 |
+----------------------+
1 row in set (0.00 sec)
```

5.1.11.4 Query 4: INOUT parameter: Increase a salary by 10%.

SQL QUERY:

PROCEDURE:

DELIMITER //

```
CREATE PROCEDURE Increase_Salary(INOUT salary_amount DECIMAL(10,2))
BEGIN
   SET salary_amount = salary_amount * 1.10;
END //
```

DELIMITER ;

SET @salary = 40000;

CALL:

CALL Increase_Salary(@salary);

DISPLAY:

SELECT @salary AS Updated_Salary;

OUTPUT:

```
+----------------+
| Updated_Salary |
+----------------+
|          40000 |
+----------------+
1 row in set (0.00 sec)
```

5.1.12 JOIN Operations

5.1.12.1 Query 1: Display employee names and department names.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      e.Last_Name,
      d.Department_Name
FROM Employee e
INNER JOIN Department d
   ON e.Department_ID = d.Department_ID;
```

OUTPUT:

```
+-------------+------------+-----------+-----------------+
| Employee_ID | First_Name | Last_Name | Department_Name |
+-------------+------------+-----------+-----------------+
|           1 | Rahul      | Sharma    | HR              |
|           2 | Priya      | Reddy     | IT              |
|           3 | Amit       | Kumar     | Finance         |
|           4 | Sneha      | Rao       | Marketing       |
|           5 | Kiran      | Reddy     | Sales           |
|           6 | Ananya     | Patel     | Operations      |
|           7 | Vikram     | Singh     | Development     |
|           8 | Divya      | Mohan     | Testing         |
|           9 | Arjun      | Verma     | Support         |
|          10 | Meena      | Das       | Administration  |
+-------------+------------+-----------+-----------------+
10 rows in set (0.00 sec)
```

5.1.12.2 Query 2: Display employee names and attendance dates.

SQL QUERY:

```
ELECT e.Employee_ID,
      e.First_Name,
```

```
    e.Last_Name,
    a.Attendance_Date,
    a.Attendance_Status
FROM Employee e
INNER JOIN Attendance a
  ON e.Employee_ID = a.Employee_ID;
```

OUTPUT:

```
    -> ON e.Employee_ID = a.Employee_ID;
+-------------+------------+-----------+-----------------+-------------------+
| Employee_ID | First_Name | Last_Name | Attendance_Date | Attendance_Status |
+-------------+------------+-----------+-----------------+-------------------+
|           1 | Rahul      | Sharma    | 2026-09-01      | Present           |
|           2 | Priya      | Reddy     | 2026-09-01      | Present           |
|           3 | Amit       | Kumar     | 2026-09-01      | Half-day          |
|           4 | Sneha      | Rao       | 2026-09-01      | Absent            |
|           5 | Kiran      | Reddy     | 2026-09-01      | Present           |
|           6 | Ananya     | Patel     | 2026-09-02      | Present           |
|           7 | Vikram     | Singh     | 2026-09-02      | Present           |
|           8 | Divya      | Mohan     | 2026-09-02      | Absent            |
|           9 | Arjun      | Verma     | 2026-09-02      | Present           |
|           1 | Rahul      | Sharma    | 2026-09-03      | Present           |
+-------------+------------+-----------+-----------------+-------------------+
10 rows in set (0.00 sec)
```

5.1.12.3 Query 3: Display employee names, basic salaries and net salaries.

SQL QUERY:

```
SELECT e.Employee_ID,
    e.First_Name,
    s.Basic_Salary,
    p.Net_Salary
FROM Employee e
INNER JOIN Salary s
  ON e.Employee_ID = s.Employee_ID
INNER JOIN Payroll p
  ON e.Employee_ID = p.Employee_ID
 AND s.Salary_ID = p.Salary_ID;
```

OUTPUT:

Employee_ID	First_Name	Basic_Salary	Net_Salary
1	Rahul	50000.00	60000.00
2	Priya	40000.00	47000.00
3	Amit	35000.00	41000.00
4	Sneha	30000.00	35000.00
5	Kiran	32000.00	37500.00
6	Ananya	28000.00	32600.00
7	Vikram	45000.00	53000.00
8	Divya	38000.00	44600.00
9	Arjun	33000.00	38500.00
10	Meena	42000.00	49400.00

10 rows in set (0.00 sec)

5.1.12.4 Query 4: Display all departments, including departments without employees.

SQL QUERY:

```
SELECT d.Department_ID,
      d.Department_Name,
      e.Employee_ID,
      e.First_Name
FROM Department d
LEFT JOIN Employee e
   ON d.Department_ID = e.Department_ID;
```

OUTPUT:

Department_ID	Department_Name	Employee_ID	First_Name
1	HR	1	Rahul
2	IT	2	Priya
3	Finance	3	Amit
4	Marketing	4	Sneha
5	Sales	5	Kiran
6	Operations	6	Ananya
7	Development	7	Vikram
8	Testing	8	Divya
9	Support	9	Arjun
10	Administration	10	Meena

10 rows in set (0.00 sec)

5.1.12.5 Query 5: Display employee names, department names and payroll net salaries.

SQL QUERY:

```
SELECT e.Employee_ID,
       e.First_Name,
       d.Department_Name,
       p.Payroll_Month,
       p.Payroll_Year,
       p.Net_Salary
FROM Employee e
INNER JOIN Department d
   ON e.Department_ID = d.Department_ID
INNER JOIN Payroll p
   ON e.Employee_ID = p.Employee_ID;
```

OUTPUT:

Employee_ID	First_Name	Department_Name	Payroll_Month	Payroll_Year	Net_Salary
1	Rahul	HR	9	2026	60000.00
2	Priya	IT	9	2026	47000.00
3	Amit	Finance	9	2026	41000.00
4	Sneha	Marketing	9	2026	35000.00
5	Kiran	Sales	9	2026	37500.00
6	Ananya	Operations	9	2026	32600.00
7	Vikram	Development	9	2026	53000.00
8	Divya	Testing	9	2026	44600.00
9	Arjun	Support	9	2026	38500.00
10	Meena	Administration	9	2026	49400.00

10 rows in set (0.00 sec)

5.1.13 Combined Subqueries and Aggregate Functions

5.1.13.1 Query 1: Find employees earning more than the average basic salary.

SQL QUERY:

```
SELECT e.Employee_ID,
       e.First_Name,
       s.Basic_Salary
FROM Employee e
JOIN Salary s
   ON e.Employee_ID = s.Employee_ID
WHERE s.Basic_Salary > (
   SELECT AVG(Basic_Salary)
```

```
   FROM Salary
);
```

OUTPUT:

```
+-------------+------------+--------------+
| Employee_ID | First_Name | Basic_Salary |
+-------------+------------+--------------+
|           1 | Rahul      |     50000.00 |
|           2 | Priya      |     40000.00 |
|           7 | Vikram     |     45000.00 |
|           8 | Divya      |     38000.00 |
|          10 | Meena      |     42000.00 |
+-------------+------------+--------------+
5 rows in set (0.01 sec)
```

5.1.13.2 Query 2: Find the department with the highest total basic salary.

SQL QUERY:

```
SELECT d.Department_ID,
      d.Department_Name,
      SUM(s.Basic_Salary) AS Total_Basic_Salary
FROM Department d
JOIN Employee e
   ON d.Department_ID = e.Department_ID
JOIN Salary s
   ON e.Employee_ID = s.Employee_ID
GROUP BY d.Department_ID, d.Department_Name
HAVING SUM(s.Basic_Salary) >= ALL (
   SELECT SUM(s2.Basic_Salary)
   FROM Employee e2
   JOIN Salary s2
      ON e2.Employee_ID = s2.Employee_ID
   GROUP BY e2.Department_ID
);
```

OUTPUT:

```
+---------------+-----------------+--------------------+
| Department_ID | Department_Name | Total_Basic_Salary |
+---------------+-----------------+--------------------+
|             1 | HR              |           50000.00 |
+---------------+-----------------+--------------------+
1 row in set (0.00 sec)
```

5.1.13.3 Query 3: Display employees whose working hours exceed the average working hours.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      a.Working_Hours
FROM Employee e
JOIN Attendance a
   ON e.Employee_ID = a.Employee_ID
WHERE a.Working_Hours > (
   SELECT AVG(Working_Hours)
   FROM Attendance
);
```

OUTPUT:

```
+-------------+------------+---------------+
| Employee_ID | First_Name | Working_Hours |
+-------------+------------+---------------+
|           1 | Rahul      |          8.00 |
|           2 | Priya      |          8.00 |
|           5 | Kiran      |          9.00 |
|           6 | Ananya     |          7.00 |
|           7 | Vikram     |          8.00 |
|           9 | Arjun      |          9.00 |
|           1 | Rahul      |          8.00 |
+-------------+------------+---------------+
7 rows in set (0.01 sec)
```

5.1.13.4 Query 4: . Display employees whose net salary is greater than the average net salary.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      p.Net_Salary
FROM Employee e
JOIN Payroll p
   ON e.Employee_ID = p.Employee_ID
WHERE p.Net_Salary > (
   SELECT AVG(Net_Salary)
   FROM Payroll
);
```

OUTPUT:

```
+-------------+------------+------------+
| Employee_ID | First_Name | Net_Salary |
+-------------+------------+------------+
|           1 | Rahul      |   60000.00 |
|           2 | Priya      |   47000.00 |
|           7 | Vikram     |   53000.00 |
|           8 | Divya      |   44600.00 |
|          10 | Meena      |   49400.00 |
+-------------+------------+------------+
5 rows in set (0.00 sec)
```

5.1.13.5 Query 5: Find the employee with the greatest number of attendance records.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      COUNT(a.Attendance_ID) AS Total_Attendance
FROM Employee e
JOIN Attendance a
   ON e.Employee_ID = a.Employee_ID
GROUP BY e.Employee_ID, e.First_Name
HAVING COUNT(a.Attendance_ID) >= ALL (
   SELECT COUNT(*)
   FROM Attendance
   GROUP BY Employee_ID
);
```

OUTPUT:

```
+-------------+------------+------------------+
| Employee_ID | First_Name | Total_Attendance |
+-------------+------------+------------------+
|           1 | Rahul      |                2 |
+-------------+------------+------------------+
1 row in set (0.01 sec)
```

5.1.14 Integrated Attendance and Payroll Queries

5.1.14.1 Query 1: Display employees who were absent on September 1, 2026.

SQL QUERY:

```
SELECT e.Employee_ID,
```

```
    e.First_Name,
    e.Last_Name,
    a.Attendance_Date,
    a.Attendance_Status
FROM Employee e
JOIN Attendance a
  ON e.Employee_ID = a.Employee_ID
WHERE a.Attendance_Date = '2026-09-01'
 AND a.Attendance_Status = 'Absent';
```

OUTPUT:

```
+-------------+------------+-----------+-----------------+-------------------+
| Employee_ID | First_Name | Last_Name | Attendance_Date | Attendance_Status |
+-------------+------------+-----------+-----------------+-------------------+
|           4 | Sneha      | Rao       | 2026-09-01      | Absent            |
+-------------+------------+-----------+-----------------+-------------------+
1 row in set (0.00 sec)
```

5.1.14.2 **Calculate total working hours for each employee.**
SQL QUERY:

```
SELECT e.Employee_ID,
    e.First_Name,
    SUM(a.Working_Hours) AS Total_Working_Hours
FROM Employee e
JOIN Attendance a
  ON e.Employee_ID = a.Employee_ID
GROUP BY e.Employee_ID, e.First_Name;
```

OUTPUT:

```
+-------------+------------+---------------------+
| Employee_ID | First_Name | Total_Working_Hours |
+-------------+------------+---------------------+
|           1 | Rahul      |               16.00 |
|           2 | Priya      |                8.00 |
|           3 | Amit       |                4.00 |
|           4 | Sneha      |                0.00 |
|           5 | Kiran      |                9.00 |
|           6 | Ananya     |                7.00 |
|           7 | Vikram     |                8.00 |
|           8 | Divya      |                0.00 |
|           9 | Arjun      |                9.00 |
+-------------+------------+---------------------+
9 rows in set (0.00 sec)
```

5.1.14.3 Query 3: Display employees who have taken more than two days of leave.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      SUM(l.Number_of_Days) AS Total_Leave_Days
FROM Employee e
JOIN Leave_Request l
   ON e.Employee_ID = l.Employee_ID
WHERE l.Approval_Status = 'Approved'
GROUP BY e.Employee_ID, e.First_Name
HAVING SUM(l.Number_of_Days) > 2;
```

OUTPUT:

```
+-------------+------------+------------------+
| Employee_ID | First_Name | Total_Leave_Days |
+-------------+------------+------------------+
|           7 | Vikram     |                3 |
|          10 | Meena      |                3 |
+-------------+------------+------------------+
2 rows in set (0.00 sec)
```

5.1.14.4 Query 4: Calculate gross salary for each employee.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      s.Basic_Salary,
      s.House_Rent_Allowance,
      s.Other_Allowances,
      (
         s.Basic_Salary
         + s.House_Rent_Allowance
         + s.Other_Allowances
      ) AS Gross_Salary
FROM Employee e
JOIN Salary s
   ON e.Employee_ID = s.Employee_ID;
```

OUTPUT:

Employee_ID	First_Name	Basic_Salary	House_Rent_Allowance	Other_Allowances	Gross_Salary
1	Rahul	50000.00	10000.00	5000.00	65000.00
2	Priya	40000.00	8000.00	3000.00	51000.00
3	Amit	35000.00	7000.00	2500.00	44500.00
4	Sneha	30000.00	6000.00	2000.00	38000.00
5	Kiran	32000.00	6400.00	2500.00	40900.00
6	Ananya	28000.00	5600.00	1800.00	35400.00
7	Vikram	45000.00	9000.00	3500.00	57500.00
8	Divya	38000.00	7600.00	2800.00	48400.00
9	Arjun	33000.00	6600.00	2200.00	41800.00
10	Meena	42000.00	8400.00	3000.00	53400.00

10 rows in set (0.00 sec)

5.1.14.5 Query 6: Calculate net salary by subtracting deductions from gross salary.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      s.Basic_Salary,
      s.House_Rent_Allowance,
      s.Other_Allowances,
      s.Deductions,
      (
         s.Basic_Salary
         + s.House_Rent_Allowance
         + s.Other_Allowances
      ) AS Gross_Salary,
      (
         s.Basic_Salary
         + s.House_Rent_Allowance
         + s.Other_Allowances
         - s.Deductions
      ) AS Net_Salary
FROM Employee e
JOIN Salary s
   ON e.Employee_ID = s.Employee_ID;
```

OUTPUT:

Employee_ID	First_Name	Basic_Salary	House_Rent_Allowance	Other_Allowances	Deductions	Gross_Salary	Net_Salary
1	Rahul	50000.00	10000.00	5000.00	3000.00	65000.00	62000.00
2	Priya	40000.00	8000.00	3000.00	2000.00	51000.00	49000.00
3	Amit	35000.00	7000.00	2500.00	1500.00	44500.00	43000.00
4	Sneha	30000.00	6000.00	2000.00	1200.00	38000.00	36800.00
5	Kiran	32000.00	6400.00	2500.00	1400.00	40900.00	39500.00
6	Ananya	28000.00	5600.00	1800.00	1000.00	35400.00	34400.00
7	Vikram	45000.00	9000.00	3500.00	2500.00	57500.00	55000.00
8	Divya	38000.00	7600.00	2800.00	1800.00	48400.00	46600.00
9	Arjun	33000.00	6600.00	2200.00	1300.00	41800.00	40500.00
10	Meena	42000.00	8400.00	3000.00	2200.00	53400.00	51200.00

10 rows in set (0.00 sec)

5.1.14.6 Query 6: **Display employees whose payroll payment status is Pending.**

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      e.Last_Name,
      p.Payroll_Month,
      p.Payroll_Year,
      p.Net_Salary,
      p.Payment_Status
FROM Employee e
JOIN Payroll p
   ON e.Employee_ID = p.Employee_ID
WHERE p.Payment_Status = 'Pending';
```

OUTPUT:

Employee_ID	First_Name	Last_Name	Payroll_Month	Payroll_Year	Net_Salary	Payment_Status
6	Ananya	Patel	9	2026	32600.00	Pending
9	Arjun	Verma	9	2026	38500.00	Pending

2 rows in set (0.00 sec)

5.1.14.7 Query 7: Rank employees according to payroll net salary.

SQL QUERY:

```
SELECT e.Employee_ID,
      e.First_Name,
      p.Net_Salary,
      RANK() OVER (
         ORDER BY p.Net_Salary DESC
      ) AS Salary_Rank
```

```
FROM Employee e
JOIN Payroll p
   ON e.Employee_ID = p.Employee_ID;
```

OUTPUT:

Employee_ID	First_Name	Net_Salary	Salary_Rank
1	Rahul	60000.00	1
7	Vikram	53000.00	2
10	Meena	49400.00	3
2	Priya	47000.00	4
8	Divya	44600.00	5
3	Amit	41000.00	6
9	Arjun	38500.00	7
5	Kiran	37500.00	8
4	Sneha	35000.00	9
6	Ananya	32600.00	10

10 rows in set (0.01 sec)

5.1.14.8 Query 8: Display the total payroll amount for each department.

SQL QUERY:

```
SELECT d.Department_ID,
      d.Department_Name,
      SUM(p.Net_Salary) AS Total_Payroll
FROM Department d
JOIN Employee e
   ON d.Department_ID = e.Department_ID
JOIN Payroll p
   ON e.Employee_ID = p.Employee_ID
GROUP BY d.Department_ID, d.Department_Name;
```

OUTPUT:

Department_ID	Department_Name	Total_Payroll
1	HR	60000.00
2	IT	47000.00
3	Finance	41000.00
4	Marketing	35000.00
5	Sales	37500.00
6	Operations	32600.00
7	Development	53000.00
8	Testing	44600.00
9	Support	38500.00
10	Administration	49400.00

10 rows in set (0.00 sec)